Programación II

Guía 1

Fundamentos de programación estructurada en C

Plataformas: Windows 10/11 y GNU/Linux
Herramientas: Visual Studio Code, GCC y GDB

Edición 2026

Propósito de esta guía

En Programación I aprendiste a resolver problemas mediante algoritmos y a implementarlos en Python. En esta primera guía práctica de Programación II comenzarás a expresar esas mismas ideas utilizando el lenguaje C.

El objetivo no es comenzar nuevamente desde cero, sino reconocer cómo se representan en C conceptos ya conocidos, incorporando progresivamente las características propias del lenguaje: declaración de variables, tipos de datos, operadores, estructuras de decisión y estructuras repetitivas.

A lo largo de los ejercicios también utilizarás las herramientas configuradas en la Guía 0 para escribir, compilar, ejecutar, analizar errores y depurar programas desde Visual Studio Code.

Resultados de aprendizaje

Al finalizar podrás:

- declarar variables y utilizar tipos de datos básicos del lenguaje C;
- ingresar y mostrar información mediante `scanf()` y `printf()`;
- utilizar operadores aritméticos, relacionales y lógicos;
- implementar estructuras de decisión mediante `if`, `else` y `switch`;
- utilizar estructuras repetitivas para resolver problemas;
- interpretar errores y advertencias producidos por el compilador;
- analizar el comportamiento de un programa antes de ejecutarlo;
- utilizar las herramientas básicas de compilación y depuración de Visual Studio Code.

Antes de comenzar

Para resolver los ejercicios de esta guía utilizarás el entorno de trabajo configurado durante la Guía 0.

Crea una carpeta para esta práctica y guarda cada ejercicio en un archivo independiente. Por ejemplo:

```
ejercicio01.c
ejercicio02.c
ejercicio03.c
...
```

Flujo de trabajo

Para cada ejercicio se recomienda seguir el mismo procedimiento:

1. analizar el problema antes de comenzar a programar;
2. crear el archivo fuente correspondiente;
3. escribir y guardar el programa;
4. compilar utilizando `Ctrl+Shift+B`;
5. leer y corregir los errores o advertencias informados por GCC;
6. ejecutar el programa desde la terminal integrada;

7. comprobar su funcionamiento utilizando diferentes datos de prueba.

Cuando sea necesario, también podrás utilizar el depurador de Visual Studio Code para observar paso a paso la ejecución del programa.

Verificación

Antes de comenzar con los ejercicios, comprueba que puedes abrir una carpeta de trabajo en Visual Studio Code, crear un archivo con extensión `.c`, compilarlo y ejecutar el programa desde la terminal integrada.

Bloque A: Entrada, salida y estructuras de decisión

En este primer grupo de ejercicios se trabajará con entrada y salida de datos, operadores y estructuras condicionales.

Ejercicio 1: Mayor de dos números

Escribir un programa que lea dos números enteros y escriba el mayor de los dos o un mensaje si son iguales.

Ejercicio 2: Par o impar

Escribir un programa que lea un número e imprima un mensaje `Par` o `Impar`.

Ejercicio 3: Operaciones aritméticas mediante if–else

Leer un carácter y dos números enteros. Si el carácter leído es un operador aritmético, calcular la operación correspondiente; si es cualquier otro, mostrar error.

Hacer el programa utilizando `if-else`.

Ejercicio 4: Operaciones aritméticas mediante switch

Hacer el mismo programa del ejercicio anterior utilizando la instrucción `switch()`.

Bloque B: Análisis y diagnóstico de programas

En los siguientes ejercicios deberás observar cuidadosamente el código, identificar errores y analizar su comportamiento.

Ejercicio 5: ¿Qué falla en los siguientes códigos?

Analiza cada uno de los siguientes programas e identifica el problema.

a)

```c
int main (void)
{
    printf ("Hola Mundo\n")
    return 0;
}
```

b)

```c
int main (void)
{
    int i;

    for (i = 0, i < 10, i++)
        printf ("%d\n", i);

    return 0;
}
```

c)

```c
int main (void)
{
    int a = 0, b = 1;

    if (a == 1)
        a++;

    b++;
    else
        b--;

    printf ("a = %d\n", a);
    printf ("b = %d\n", b);

    return 0;
}
```

d)

```c
int main (void)
{
    int i;

    for (i = 0; i < 10; i++)
        printf ("%d\n", I);

    return 0;
}
```

Ejercicio 6: ¿Qué imprimen los siguientes códigos?

Antes de ejecutar los programas, analiza cada código e indica qué resultado esperas obtener.

Luego puedes comprobar tu respuesta compilando y ejecutando cada caso.

a)

```c
int main (void)
{
    int i;

    for (i = 0; i < 10; i++);

    printf ("%d\n", i);

    return 0;
}
```

b)

```c
int main (void)
{
    int a = 0, b = 1;

    if (a == 1)
        a++;

    b++;

    printf ("a = %d\n", a);
    printf ("b = %d\n", b);

    return 0;
}
```

c)

```c
int main (void)
{
    int i;

    for (i = 0; i > 10; i++) {

        printf ("%d\n", i);

        if (i == 4)
            break;
    }

    return 0;
}
```

d)

```c
int main (void)
{
    int i;

    for (i = 0; i < 10; i++) {

        if (i < 5)
            continue;

        printf ("%d\n", i);
    }

    return 0;
}
```

Bloque C: Iteraciones y operadores

En este bloque se trabajará con estructuras repetitivas y operadores del lenguaje C.

Ejercicio 7: Serie de Fibonacci

Generar los N primeros términos de la serie de Fibonacci.

El valor de N (entero y positivo) deberá ser leído por teclado.

En esta serie los dos primeros números son 1, y el resto se obtiene sumando los dos anteriores:

$$1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21, \ldots$$

Ejercicio 8: Conversión de temperaturas

Realizar un programa en C que escriba una tabla de dos columnas para la conversión entre las temperaturas en grados Fahrenheit, comprendidas entre 0 °F y 300 °F, según incrementos de 20 °F, y su equivalente en grados Celsius.

La conversión entre grados Celsius y grados Fahrenheit obedece a la fórmula:

$$°\mathrm{C} = \frac{5\,(°\mathrm{F} - 32)}{9}$$

siendo °C la temperatura en grados Celsius y °F la temperatura en grados Fahrenheit.

Ejercicio 9: Operadores aritméticos

Realizar un programa que lea dos números y aplique sobre ellos todos los operadores aritméticos, incluyendo el pre y post incremento y decremento.

Ejercicio 10: Lectura repetitiva de números

Escribir un programa que:

a. pida por teclado un número (dato entero);

b. repita el paso anterior mientras que el número introducido sea distinto de cero;

c. muestre cuántos números mayores que cero han sido introducidos por el usuario, así como la suma de todos ellos.

Bloque D: Ejercicios adicionales – Ingeniería en Telecomunicaciones

Los siguientes ejercicios aplican los conceptos anteriores a situaciones relacionadas con sistemas de telecomunicaciones.

Ejercicio 11: Intensidad de señal

Una estación recibe tres mediciones de la intensidad de señal de una antena, expresadas en dBm.

Escriba un programa en C que:

a. lea los tres valores;

b. calcule el promedio;

c. determine si la señal es:

   i. Debil: menor a -85 dBm.

   ii. Aceptable: entre -85 y -60 dBm.

   iii. Fuerte: mayor a -60 dBm.

Ejemplo:

```
Ingrese intensidad 1: -90
Ingrese intensidad 2: -80
Ingrese intensidad 3: -85

Promedio: -85.00 dBm -> Senal debil
```

Ejercicio 12: Latencia de una conexión

Una aplicación realiza tres pruebas de latencia, expresadas en milisegundos.

Leer los tres valores y mostrar:

a. el promedio;

b. si alguna prueba supera los 150 ms, mostrar `Conexión inestable`. Caso contrario, mostrar `Conexión estable`.

Ejemplo:

```
Ingrese latencia 1 (ms): 100
Ingrese latencia 2 (ms): 220
Ingrese latencia 3 (ms): 130

Promedio de latencia: 150.00 ms
Conexion inestable
```

Lista de comprobación final

Antes de finalizar la guía, comprueba que puedes:

☐ crear y guardar un programa en C desde Visual Studio Code;

☐ compilarlo utilizando la tarea configurada en la Guía 0;

☐ interpretar errores y advertencias informados por GCC;

☐ ejecutar el programa desde la terminal integrada;

☐ ingresar y mostrar datos utilizando `scanf()` y `printf()`;

☐ utilizar estructuras de decisión;

☐ utilizar estructuras repetitivas;

☐ analizar código antes de ejecutarlo;

☐ probar un programa utilizando diferentes valores de entrada.